
WHATSAPP INFRASTRUCTURE GUIDE

בניית WhatsApp Hub עצמאי

מדריך מלא – מאפס ועד API חי שמדבר עם WhatsApp,
כולל אבטחה, חשיפה לאינטרנט ואינטגרציה לכל פלטפורמה

🕒 זמן ביצוע: ~45 דקות • €3.79 לחודש • יכולת בלתי מוגבלת

מחבר

Noam Nissan

תאריך

מאי 2026

github.com/Noam13-w/wa-hub-demo

בניית WhatsApp Hub עצמאי – מדריך מלא

מה תקבל בסוף המדריך הזה: REST API מאובטח שמדבר עם WhatsApp, רץ על שרת שלך, חשוף לאינטרנט בצורה הגיונית, ומדבר עם כל פלטפורמה שאתה רוצה – Base44, Bubble, Firebase, Make, Python, או כל דבר שיוזע .HTTP.

זמן ביצוע: 45~ דקות בידיים. **עלות:** €3.79–€3.99 לחודש (Hetzner CAX11 או CX23). **רישיון:** קוד מקור: MIT. github.com/Noam13-w/wa-hub-demo

⚠ הבהרה – חשוב לקרוא לפני שמתקינים. הפרויקט אינו רשמי ואינו מסונף ל-WhatsApp או ל-Meta. הוא משתמש בספרייה הלא-רשמית [Baileys](#) ומתחבר על-ידי התחזות ל-"מכשיר מקושר", מה שעלול להפר את תנאי השימוש של WhatsApp ולגרום לחסימת המספר לפי שיקול דעתה של Meta – במיוחד בשליחה המונית או לא-מבוקשת. מסופק "כמות שהוא", ללא אחריות. האחריות כולה עליכם – לשלוח רק למי שנתן הסכמה מפורשת מראש, ולעמוד בכל דין (GDPR, חוק הספאם סעיף 30א, ועוד). אינו ייעוץ משפטי. ראו [DISCLAIMER.md](#).

תוכן העניינים

1. מה אנחנו בונים?
2. רשימת מה שצריך לפני שמתחילים
3. שלב 1 – קניית שרת Hetzner
4. שלב 2 – חיבור SSH ראשון
5. שלב 3 – בוחרים מסלול
6. שלב 4 – מסלול A: התקנה ידנית
7. שלב 4 חלופי – מסלול B: עם Claude Code
8. שלב 5 – פיירינג WhatsApp
9. שלב 6 – חשיפה לאינטרנט עם Cloudflare Tunnel
10. שלב 7 – שימוש מכל מערכת
11. שלב 8 – חיבור דרך מערכת vibe-coding
12. שלב 9 – שליחה בטוחה: מניעת חסימה
13. שלב 10 – אבטחה לפרודקשן
14. שלב 11 – חיבור מודל AI
15. טיפים מהשטח
16. פתרון בעיות נפוצות
17. שאלות נפוצות
18. נספח – מדריך חירום מהיר

מה אנחנו בונים?



הטלפון שלך

WhatsApp

pairing n-QR



Hetzner Server

€3.79 / mo

wa-hub-demo (Node 20)

└ Baileys

└ REST :3060

└ WS :3061

└ Webhook → HMAC

חוסם wtf · בלבד loopback



כל אפליקציה

Base44

Bubble · Webflow

Firebase

Make · n8n · Zapier

Python · PHP

Apps Script

1. הטלפון פותח חיבור מוצפן לשרת (WhatsApp protocol).
2. שוטף את הפרוטוקול ב-REST API. wa-hub-demo השרת מריץ
3. אפליקציות חיצוניות מדברות עם ה-API בתעבורה מוצפנת דרך Cloudflare Tunnel.

מה זה כן:

- **HTTP API פשוט** מעל פרוטוקול POST WhatsApp Web. לשליחה, Webhook לקבלה. כלום מסובך.
- **בעלות מלאה** – הקוד שלך, השרת שלך, ה-token שלך, ההודעות שלך.
- **עלות קבועה** – €3.79 לחודש כל החודש, בלי קשר אם שלחת 10 או מיליון הודעות.

- קוד פתוח (MIT) – תפרק, תשנה, תפרסם, תמכור.

הקוד שתתקין

הפרויקט `wa-hub-demo` הוא קוד פתוח, זמין ב-github.com/Noam13-w/wa-hub-demo. הוא כולל:

- שכבת Baileys** שמטפלת בפרוטוקול WhatsApp Web (חיבור, פיירינג, התחברות מחדש אוטומטית, נירמול הודעות)
- REST API** ב-Express עם מעל 20 endpoints – שליחת טקסט/תמונה/קובץ/אודיו/מיקום/ריאקציה, פיירינג, סטטוס, ניהול webhook, ניהול קבוצות, בדיקה אם מספר רשום ב-WhatsApp
- WebSocket server** שמשדר אירועים בזמן אמת (`message.incoming`, `message.outgoing`, `message.status`) עם וי כחול
- Outbound webhooks** עם חתימת HMAC-SHA256 לאימות
- Bearer auth** עם השוואה constant-time + rate-limit
- טיפול ב-LID** של 7 Baileys+ (שדות `senderPn` / `participantPn` / `remoteJidAlt` כדי לחלץ מספר טלפון אמיתי)
- systemd unit מוקשח** עם `NoNewPrivileges`, `ProtectSystem=strict`, `CapabilityBoundingSet=`, ריק, `MemoryMax=512M`
- תיעוד API מלא** ב-`docs/API.md` ו-`docs/ARCHITECTURE.md`

הכל ב-MIT, אתה יכול לעשות איתו מה שתרצה.

מה זה לא:

- לא רשמי מ-Meta**. WhatsApp לא מספקים API ישיר כזה. אנחנו מתחזים ל-Linked Device. זה עובד כי הפרוטוקול גלוי, אבל אם תעבירו לקוד שלכם 10K הודעות ביום של ספאם – WhatsApp יחסום את המספר. השתמשו רק להודעות שמישהו ביקש.
- לא Multi-tenant out-of-the-box**. מספר אחד = שרת אחד = instance אחד. אם רוצים לתת שירות לכמה לקוחות, כל לקוח מקבל instance נפרד (ראו "צעדים הבאים").
- לא Plug-and-play של Meta WhatsApp Cloud API**. זה משהו אחר – של Meta, דורש אישור עסקי, וכרוך ב-billing per-message. שני העולמות לא חופפים.

למי המדריך הזה:

- מפתחים** שרוצים שליטה מלאה ב-stack של WhatsApp שלהם
- No-code builders** (Base44, Bubble, Webflow) שרוצים לחבר WhatsApp בלי לקנות תוסף
- אנשי אוטומציה** (Make, n8n, Zapier) שצריכים אינטגרציה מינימלית
- חברות קטנות** שמעדיפות לשלם €3.79 לחודש קבוע במקום \$0.05 לכל הודעה

רשימת מה שצריך לפני שמתחילים

- ☐ כרטיס אשראי (Hetzner גובה €3.79 בחודש הראשון)
- ☐ טלפון עם WhatsApp פעיל (להוסיף כ-Linked Device)
- ☐ חשבון GitHub (חינם, לשכפול הקוד)
- ☐ חשבון Cloudflare (חינם, אופציונלי – ל-Tunnel)
- ☐ טרמינל (Mac/Linux: בנוי פנימי. Windows: PowerShell או WSL)

שלב 1 – קניית שרת Hetzner

1.1 הרשמה

לכו ל-hetzner.com/cloud ולחצו "Sign Up".

למה Hetzner? היחס מחיר/ביצועים הכי טוב באירופה. €3.79 לחודש = vCPU + 4GB RAM + 40GB SSD + 20TB 2 = תעבורה. שווה ערך ל-AWS t4g.medium ב-\$25 לחודש.

1.2 יצירת פרויקט וחיוב

אחרי האימות:

1. הקליקו "New Project +" → תנו שם (למשל `wa-hub`)
2. בתפריט שמאל → "Billing" → הכניסו פרטי כרטיס
3. בתפריט שמאל → "Security" → "SSH Keys" → "Add SSH Key"

1.3 יצירת מפתח SSH (אם אין לך)

הריצו פקודה אחת בלבד:

על Mac / Linux / WSL:

BASH
העתק

```
ssh-keygen -t ed25519 -C "my-laptop"
```

על Windows PowerShell:

POWERSHELL
העתק

```
ssh-keygen -t ed25519 -C "my-laptop"
```

חשוב: הקלידו את הפקודה **לבד**, בלי להעתיק שורות אחרות אחריה. הפקודה תשאל אתכם 3 שאלות אינטראקטיביות, ואם תדביקו עוד שורה – היא תיכנס בטעות כתשובה לאחת מהן (זה קורה הרבה).

הפקודה תשאל אתכם 3 שאלות. **לכל אחת לחצו Enter** (כל ברירות המחדל בסדר):

1. Enter file in which to save the key → Enter (ברירת מחדל: `~/.ssh/id_ed25519`)
2. Enter passphrase (empty for no passphrase) → Enter (בלי סיסמה למפתח)
3. Enter same passphrase again → Enter (אישור)

passphrase? היא מצפינה את המפתח הפרטי על הדיסק (ותצטרכו להקליד אותה בכל חיבור). אם המחשב הוא רק שלכם – Enter פעמיים (בלי passphrase) זה בסדר גמור.

עכשיו **בנפרד** הציגו את המפתח הציבורי:

BASH
העתק

```
cat ~/.ssh/id_ed25519.pub
```

על Windows PowerShell

POWERSHELL
העתק

```
Get-Content $HOME\.ssh\id_ed25519.pub
```

העתיקו את כל השורה שהודפסה (מתחילה ב- `ssh-ed25519`) → הדביקו ב-Hetzner → תנו שם זיהוי (למשל `laptop`) → שמור.

-C זה רק תווית לזיהוי המפתח – לא חובה ולא מוסיף אבטחה. תנו שם שיעזור לכם לזהות אותו (`my-laptop` , `office-mac`).

1.4 הזמנת השרת

1. בתפריט שמאל → "Servers" → "Add Server +"

2. **Location:** Falkenstein (גרמניה – הכי קרוב לישראל מבחינת ping, ~60ms). אם אין שם – Nuremberg גם גרמניה ועובד באותה איכות.

3. **Image:** Ubuntu 24.04 LTS (גם LTS 26.04 עובד, אבל היא חדשה מ-2026/04 ופחות בדוקה. 24.04 מספיקה ל-5 שנים קדימה ונבדקה במדריך הזה לעומק).

4. **Type:** בחרו אחד משני אלה (שניהם עובדים זהה – תלוי במה שזמין באתר באותו רגע):

- **CX23** (x86 / Intel-AMD) – €3.99 לחודש · זמין בכל המיקומים
- **CAX11** (ARM / Ampere) – €3.79 לחודש · זמין רק במיקומים מסוימים (כרגע Nuremberg / Helsinki). אם הטאב "Arm64 (Ampere)" קיים – מצוין, אחרת השתמשו ב-CX23.

5. **Networking:** השאירו ברירת מחדל (IPv4 + IPv6)

6. **SSH keys:** סמנו את המפתח שהוספתם

7. **Volumes / Firewalls / Backups:** דלגו (לא צריך)

8. **Name:** `wa-hub-demo` (או מה שתמצאו)

9. לחצו "Create & Buy now"

תוך 30 שניות השרת יהיה מוכן. שימו לב לכתובת ה-IPv4 – תזדקקו לה.

CX23 vs CAX11 – מה ההבדל בפועל? אפס מבחינת הקוד שלנו. שניהם מריצים Node 20 בלי הבדל. CAX11 קצת יותר יעיל אנרגטית ו-€0.20 זול יותר. CX23 קצת יותר זמין ובמיקומים יותר. **קחו את מה שזמין במיקום הקרוב אליכם – זה לא משנה.** הזמינות של ARM משתנה מדי פעם כי Hetzner מנהלים מלאי לפי דרישה.

שלב 2 – חיבור SSH ראשון

זה הצעד שהרבה אנשים נתקעים בו בפעם הראשונה. נעבור לאט. **לא תצטרכו להדביק את המפתח שלכם בשום מקום בשלב הזה** – כבר הדבקנו אותו ב-Hetzner בשלב 1.3, וזה מה שמספיק. ה-SSH client במחשב שלכם ימצא לבד את המפתח הפרטי ב- `~/.ssh/id_ed25519` וישתמש בו.

2.1 – מציאת ה-IP של השרת

בפאנל של Hetzner Cloud → השרת שלכם → תחת "IPv4" תראו כתובת כמו `203.0.113.42`. **העתיקו אותה** – תזדקקו לה בעוד רגע.

2.2 – פתיחת טרמינל

- **Mac:** Cmd+Space → הקלידו `Terminal` → Enter
 - **Linux:** Ctrl+Alt+T (תלוי בהפצה)
 - **Windows:** Start → הקלידו `PowerShell` → Enter
- Windows 10/11 כולל את `ssh` מובנה. לא צריך להתקין כלום.

2.3 – חיבור

הקלידו (החליפו את `203.0.113.42` ב-IP שלכם):

BASH
העתק

```
ssh root@203.0.113.42
```

בפעם הראשונה תופיע הודעה:

העתק
The authenticity of host '203.0.113.42 (...)' can't be established.
ED25519 key fingerprint is SHA256:..
Are you sure you want to continue connecting (yes/no/[fingerprint])?

הקלידו `yes` ו-Enter. זאת רק שאלה חד-פעמית – SSH אומר "לא ראיתי את השרת הזה בעבר, אתה בטוח שזה הוא?"

2.4 – אתם בפנים

אם הכל בסדר, תראו:

העתק
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-XX-generic x86_64)
root@wa-hub-demo:~#

הסימן `#` בסוף השורה אומר שאתם מחוברים כ-`root` לשרת. כל מה שתקלידו מכאן ירוץ **על השרת בגרמניה**, לא על המחשב שלכם.

■ 2.5 – בדיקה זריזה שהשרת חי

BASH
העתק

```
lsb_release -d
```

BASH
העתק

```
free -h
```

BASH
העתק

```
curl -fsSL ifconfig.me; echo
```

הראשונה אומרת מה גרסת Ubuntu, השנייה כמה זיכרון, השלישית מאשרת שיש אינטרנט (היא מדפיסה את ה-IP שלכם – אותו אחד שהדבקתם למעלה).

■ 2.6 – אם לא הצליח להתחבר

▼ הודעה: Permission denied (publickey)

זה אומר שה-SSH key לא נמצא או לא תקין:

- ודאו שב-Hetzner סימנתם את ה-SSH key בזמן יצירת השרת. אם שכחתם → צריך לאפס.
- ודאו שיצרתם את המפתח עם `ssh-keygen` (שלב 1.3) ושהמפתח קיים: `ls ~/.ssh/id_ed25519`
- (Mac/Linux) או `ls $HOME\.ssh\id_ed25519` (PowerShell).
- אם המפתח קיים והתקלה ממשיכה – חכו 30 שניות (לפעמים Hetzner צריך זמן להזריק את המפתח לשרת) ותנסו שוב.

שנחתם להוסיף SSH key בזמן יצירת השרת

מהפאנל של Hetzner:

- 1. כנסו לשרת → "Rescue" → אפסו את סיסמת root
- 2. תיכנסו עם הסיסמה שקיבלתם: ssh root@ והקלידו את הסיסמה
- 3. הדביקו את המפתח הציבורי שלכם לקובץ ~/.ssh/authorized_keys :

BASH העתק

```
mkdir -p ~/.ssh
echo "ssh-ed25519 AAAA... my-laptop" >> ~/.ssh/authorized_keys
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

(החליפו את ssh-ed25519 AAAA... בתוכן האמיתי של ~/.ssh/id_ed25519.pub שלכם).
מהפעם הבאה – תוכלו להיכנס עם המפתח, בלי סיסמה.

הודעה: Connection timed out או Connection refused

- ודאו שהשרת באמת רץ בפאנל של Hetzner (אם הוא בסטטוס "starting" – חכו דקה)
- ודאו שלא העתקתם IP לא נכון (IPv6 נראה אחרת – צריך IPv4)
- אם אתם ברשת מוגבלת (אוניברסיטה, חברה) – פורט 22 לפעמים חסום משם

שלב 3 – בוחרים מסלול

מכאן יש שלוש דרכים לבנות את ה-Hub:

מסלול A – ידני	מסלול B – Claude Code	מסלול C – Express
קצב	מהיר	מידי (3 דקות)
למידה	מבינים את הזרימה	אפס – קופסה שחורה
התאמה אישית	קלה (מבקשים מ-Claude)	אחר-כך לערוך ידנית
מומלץ ל-	פעם שלישית והלאה	פרודקשן שאתה סומך עליו

המלצה: פעם ראשונה – לכו על מסלול A (ידני) כדי להבין כל שלב. כשאתם כבר מכירים את התהליך – מסלול C (שורה אחת) הוא הדרך המהירה ל-deploy.

■ מסלול Express – C (אופציה אקספרסית: שורה אחת)

אם יש לכם שרת Hetzner חדש (Ubuntu 24.04 כ-root), והעיקר זה שזה יעבוד מהר:

BASH
העתק

```
curl -fsSL https://raw.githubusercontent.com/Noam13-w/wa-hub-demo/main/deploy/install.sh | bash
```

הסקריפט עושה הכל אוטומטית:

- עדכוני מערכת + הקשחת SSH + ufw + fail2ban
 - Node 20 + יצירת npm install + git clone + user `wahub`
 - יצירת סודות אקראיים (`HUB_TOKEN` , `WEBHOOK_SECRET`)
 - התקנת `wa-hub.service` + drop-in לתיקון seccomp של Node 20
 - Cloudflare Tunnel + systemd unit
 - מדפיס לבסוף: URL **ציבורי**, `HUB_TOKEN`, `WEBHOOK_SECRET`, ופקודות מוכנות לפיירינג + שליחה ראשונה
- זמן ביצוע: ~3 דקות. אחרי שהוא רץ, קופצים לשלב 5 (פיירינג).

רוצים לראות מה הסקריפט עושה לפני שאתם רצים? הקובץ נמצא ב- `deploy/install.sh` ברפּו – 130 שורות קריאות, אפס קסם.

■ שלב 4 – מסלול A: התקנה ידנית

כל הפקודות מתחת מורצות **על השרת** (אחרי `ssh root@`). במקום עשרה שלבים קטנים, חילקתי לשלושה בלוקים גדולים שכל אחד עומד בפני עצמו.

■ A.1 – הקשחת השרת

עדכונים + נעילת SSH + firewall + fail2ban בריצה אחת. ~3 דקות.

BASH העתק

```
# עדכונים
apt-get update && apt-get -y dist-upgrade && apt-get -y autoremove

# SSH – בלי סיסמאות, בלי מפתחות בלבד, בלי סיסמאות
sed -i 's/^#*PasswordAuthentication .*/PasswordAuthentication no/' /etc/ssh/sshd_config
sed -i 's/^#*PermitRootLogin .*/PermitRootLogin prohibit-password/' /etc/ssh/sshd_config
sshd -t && systemctl reload ssh

# Firewall – רק SSH לאינטרנט
ufw allow 22/tcp comment 'SSH'
ufw default deny incoming && ufw default allow outgoing
yes | ufw enable

# fail2ban – חוסם IPs שמנסים brute-force
apt-get install -y fail2ban
systemctl enable --now fail2ban
```

מה השגנו: שרת שאי אפשר להיכנס אליו בלי המפתח הפרטי שלך, ובוטים שמנסים ננעלים אוטומטית אחרי 3 ניסיונות. ה-Hub עצמו לא חשוף – נטפל בזה דרך Tunnel בשלב 6.

לא להיבהל מה-output: הבלוק הזה מדפיס הרבה מאוד שורות (apt) רושם כל חבילה שמתקנת/משדרגת). תוך כדי fail2ban תראו SyntaxWarning: invalid escape sequence '\s' – אלה אזהרות קוסמטיות של Python ולא משפיעות על שום דבר. אם בסוף יש לכם prompt root@wa-hub-demo:~# – הכל בסדר. אם בנוסף ראייתם Pending kernel upgrade! – זה אומר שיש קרנל חדש שיתפוס באתחול הבא, ואין צורך לעשות עם זה כלום עכשיו.

■ A.2 – התקנת Node, יצירת משתמש שירות, ושכפול הקוד

BASH העתק

```
# Node.js 20
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt-get install -y nodejs

# משתמש שירות (בלי --create-home, כי git clone ייצור את התיקייה)
useradd --system --shell /usr/sbin/nologin --home-dir /srv/wa-hub-demo wahub

# שכפול הקוד (-) root/srv אינה ניתנת לכתיבה ל-wahub, ואז העברת בעלות
cd /srv
git clone https://github.com/Noam13-w/wa-hub-demo.git
chown -R wahub:wahub /srv/wa-hub-demo

# התקנת חבילות (-) wahub
sudo -u wahub bash -c "cd /srv/wa-hub-demo && npm install --omit=dev"
```

השתמשנו ב- `npm install` כי הוא סלחני יותר עם transitive dependencies (כמו `sharp` שתלוי ב-Baileys). אם תעדיף את ההתנהגות הקפדנית של `npm ci`, ודא שה- `package-lock.json` עדכני: `npm install` פעם אחת מקומית, push, ואז בשרת תוכל להשתמש ב- `npm ci`.

למה לא `--create-home` ? הדגל יוצר את `/srv/wa-hub-demo` ריק, וגיט אחר-כך מסרב לעשות clone לתיקייה לא ריקה. הפתרון: יוצרים את המשתמש בלי תיקייה, גיט בונה את התיקייה במהלך ה-clone כ-root, ואז משייכים אותה ל-wahub.

BASH העתק

```
# יצירת סודות אקראיים (256 ביט)
HUB_TOKEN=$(openssl rand -hex 32)
WEBHOOK_SECRET=$(openssl rand -hex 32)

# הצג אותם פעם אחת – שמור במנהל סיסמאות
echo "==== שמרו את הערכים האלה ====="
echo "HUB_TOKEN=$HUB_TOKEN"
echo "WEBHOOK_SECRET=$WEBHOOK_SECRET"
echo "===== "

# כתיבת .env
cat > /srv/wa-hub-demo/.env <<EOF
HUB_NAME=wa-hub
HUB_TOKEN=$HUB_TOKEN
WEBHOOK_SECRET=$WEBHOOK_SECRET
HUB_PORT=3060
WS_PORT=3061
RATE_LIMIT_PER_MIN=120
DATA_DIR=/srv/wa-hub-demo/data
LOG_LEVEL=info
EOF
chown wahub:wahub /srv/wa-hub-demo/.env
chmod 600 /srv/wa-hub-demo/.env

# יצירת תיקיית data
mkdir -p /srv/wa-hub-demo/data
chown -R wahub:wahub /srv/wa-hub-demo/data

# התקנה כ- systemd service, מתאושש אוטומטית)
install -m 644 /srv/wa-hub-demo/deploy/wa-hub.service /etc/systemd/system/wa-hub.service
systemctl daemon-reload
systemctl enable --now wa-hub.service

# בדיקה אחרונה – אמור להחזיר {"ok":true, "connection":"qr"}
sleep 4
curl -sS http://127.0.0.1:3060/healthz
```

השירות פעיל! systemd יחזיר אותו תוך 5 שניות אם הוא נופל, רץ עם `MemoryMax=512M` כדי לוודא שגם דליפת זיכרון של Baileys לאורך זמן לא תפיל את השרת, ויפעל אוטומטית בכל boot.

שלב 4 חלופי – מסלול B: עם Claude Code

אם רוצים לדלג על כל הטרחה ולתת ל-AI לעשות:

■ B.1 – התקנת Claude Code על המחשב שלכם

BASH העתק

```
npm install -g @anthropic-ai/claude-code
claude login
```

■ B.2 – תכנסו לשרת שלכם דרך Claude

חשבו על זה כשני טרמינלים שעובדים במקביל: אחד SSH לשרת, השני Claude Code על המחשב שלכם.

על המחשב המקומי:

BASH העתק

```
mkdir wa-hub-prep && cd wa-hub-prep
claude
```

ובתוך Claude הקלידו:

המקור,  המקור

המטרה שלי: על שרת Hetzner חדש (x86 – Ubuntu 24.04 או X.X.X.X IP, **התקן**), להתקין את הפרויקט <https://github.com/Noam13-w/wa-hub-demo>, להריץ אותו כ-`systemd service` תחת משתמש `wahub`, ולהקים Cloudflare Quick Tunnel, שיחשוף אותו לאינטרנט.

תעבוד עם Bash דרך `ssh root@X.X.X.X`. תאשר איתי כל שלב לפני שאתה מבצע.
תתחיל באודיש מצב השרת.

Claude Code יעבור צעד-צעד, ישאל לאישור לפעולות risky, וידפיס בסוף את ה-token וה-URL הציבורי. בערך **10 דקות**.

מאיפה Claude יודע איך? הוא קורא את הקוד והמדריך הזה. הוא לא מנחש – הוא רואה את אותם השלבים שאתם רואים, ומבצע אותם.

שלב 5 – פיירינג WhatsApp

ה-Hub רץ, אבל הוא עוד לא מחובר לאף מספר. עכשיו נחבר.

🕒 **זה החלק הטריקי:** ל-QR יש תוקף של **60 שניות** ואז הוא מתחלף. לכן צריך **להכין הכל מראש** – טלפון פתוח, חלון PowerShell מוכן – ואז בלחיצה אחת על השרת כל השאר רץ במהירות. הסיקוונס המלא: ~10 שניות.

■ 5.1 – להכין הכל מראש (לפני שמייצרים QR!)

(א) על הטלפון: פתח WhatsApp → הגדרות → מכשירים מקושרים → קישור מכשיר. המצלמה תפתח ותחכה למשהו לסרוק. השאר את זה פתוח.

(ב) על המחשב המקומי: פתח חלון PowerShell חדש (לא בשרת!). הקלד את הפקודה הבאה אבל אל תלחץ Enter עדיין – נריץ אותה ברגע הנכון:

POWERSHELL
העתק

```
scp root@<IP>:/tmp/qr.png $HOME\qr.png; Start-Process $HOME\qr.png
```

(החלף את – בכתובת השרת שלך. המקבילה ל-Mac/Linux:

```
(. scp root@:/tmp/qr.png ~/qr.png && open ~/qr.png
```

(ג) בחלון ה-SSH לשרת, הקלד גם פה אבל אל תלחץ Enter עדיין:

BASH
העתק

```
curl -sS -H "Authorization: Bearer $TOKEN" http://127.0.0.1:3060/api/instance/qr.png -o /tmp/qr.png
```

(וודא שעדיין יש לך \$TOKEN טעון. אם פתחת SSH מחדש, הרץ קודם:

```
( TOKEN=$(grep HUB_TOKEN /srv/wa-hub-demo/.env | cut -d= -f2)
```

■ 5.2 – הסיקוונס המהיר

עכשיו שהכל מוכן:

1. בחלון ה-SSH → Enter (curl שומר את ה-QR ב- /tmp/qr.png)

2. בחלון PowerShell → Enter (scp מוריד, ו-Start-Process פותח את התמונה)

3. בטלפון → סרוק את התמונה שנפתחה במחשב

תוך 2-3 שניות תראה בלוג של השרת **WhatsApp connected**, ובטלפון תראה את ההתקן בליסט "מכשירים מקושרים".

אם פספסת ב-60 שניות: פשוט תריץ שוב את אותן 2 פקודות (Enter ב-SSH → Enter ב-PowerShell). ה-Hub מייצר QR חדש כל 60 שניות אוטומטית.

■ 5.3 – בדיקה: שולחים הודעה ראשונה

שלחו הודעת בדיקה **למספר שני שלכם** (טלפון נוסף שברשותכם, או של חברה/שאישר/ה לקבל). על השרת – החליפו את 972500000000 במספר היעד:

BASH
העתק

```
TOKEN=$(grep HUB_TOKEN /srv/wa-hub-demo/.env | cut -d= -f2)
curl -X POST -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"to":"972500000000","text":"שבנתי Hub WhatsApp-שלום מה! 🚀"}' \
http://127.0.0.1:3060/api/messages/send/text
```

איך יודעים שזה עבד? התשובה מה-API היא אות ההצלחה – אתם אמורים לקבל JSON כזה:

```
{"ok":true,"id":"3EB0...","to":"972500000000@.whatsapp.net","timestamp":1730000000000}
```

. "ok":true עם id = ההודעה יצאה מה-Hub. תוך שנייה-שתיים היא תופיע גם בטלפון היעד.

פורמט המספר: קוד מדינה בלי + ובלי 0 בהתחלה, ואז המספר. דוגמה – 0585802298 הופך ל- 972585802298.

⚠ **שלחו רק למספרים שהסכימו לקבל מכם הודעות.** שליחה למספרים אקראיים היא הדרך המהירה ביותר שהמספר שלכם ייחסם (ראו פרק "שליחה בטוחה: מניעת חסימה").

5.4 – איפוס אם ניתקתם בטעות (או בכוונה)

אם ניתקתם את המכשיר מהטלפון (WhatsApp → מכשירים מקושרים → לחיצה על ההתקן → ניתוק), ה-Hub יזהה `LoggedOut` ויפסיק להתחבר מחדש (התנהגות מכוונת – לא רוצים שהוא ינסה לחזור בלי הסכמתכם). תראו בלוג:

```
ERROR Device was logged out from the phone. Clear /data/auth and re-pair.
```

לאפס ולקבל QR חדש:

BASH
העתק

```
TOKEN=$(grep HUB_TOKEN /srv/wa-hub-demo/.env | cut -d= -f2)
curl -X POST -H "Authorization: Bearer $TOKEN" http://127.0.0.1:3060/api/instance/logout
```

זה מוחק את `data/auth`, מפעיל מחדש את Baileys, ותוך 3-4 שניות יש QR חדש. אחר-כך חזרו לשלב 5.1 לסריקה מחדש.

למה הם לא מתחברים אוטומטית? במכוון. אם הטלפון הראשי החליט שלא רוצה את ההתקן הזה יותר, רוצים

שתאשרו במודע שזה היה תאונה ולא מישו חיצוני שהשתלט. אותו דבר קורה במקרה של `connectionReplaced` (קוד 440) – שני sessions לא מתנגשים זה עם זה אינסוף.

שלב 6 – חשיפה לאינטרנט עם Cloudflare Tunnel

עד עכשיו ה-API רץ רק מקומית (127.0.0.1). Base44 / כל אפליקציה חיצונית לא יכולה להגיע. בואו ננתב את זה החוצה – בלי לפתוח פורט בכלל.

6.1 – התקנת cloudflared

BASH
העתק

```
ARCH=$(dpkg --print-architecture) # מחזיר אוטומטית amd64 ל-CX23, arm64 ל-CAX11
curl -fsSL -o cloudflared.deb \
  "https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-
  linux-$ARCH.deb"
dpkg -i cloudflared.deb
cloudflared --version
```


■ 6.2 Quick Tunnel (URL רנדומלי, ללא חשבון)

הכי קל לדמו:

BASH
העתק

```
cloudflared tunnel --url http://127.0.0.1:3060
```

תוך 5 שניות תראו:

העתק

```
Your Quick Tunnel has been created! Visit it at (it may take some time to be reachable):  
https://soft-clouds-rapidly-eat.trycloudflare.com
```

זה ה-URL הציבורי שלכם. בדקו מחדש אחר:

BASH
העתק

```
curl https://soft-clouds-rapidly-eat.trycloudflare.com/healthz
```

חיסרון Quick Tunnel: ה-URL רנדומלי ומשתנה בכל restart. אם השרת נופל ועולה – צריך לעדכן את ה-URL בכל מקום שמשתמש בו. **מצוין לדמו**, פחות לפרודקשן.

■ 6.3 Quick Tunnel כ-systemd service (לא ייפול)

כדי שה-Tunnel יישאר חי גם אחרי שהטרמינל נסגר:

```
cat > /etc/systemd/system/cloudflared-wahub.service <<'EOF'
[Unit]
Description=Cloudflare Quick Tunnel for wa-hub
After=network-online.target wa-hub.service
Wants=network-online.target

[Service]
Type=simple
ExecStart=/usr/bin/cloudflared tunnel --no-autoupdate --url http://127.0.0.1:3060
Restart=on-failure
RestartSec=5
StandardOutput=journal
StandardError=journal
DynamicUser=yes

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now cloudflared-wahub.service

# המתינו 6 שניות, אז שלפו את ה-URL מהלוג:
sleep 6
journalctl -u cloudflared-wahub.service -n 30 --no-pager | \
  grep -Eo 'https://[a-z0-9-]+\.\trycloudflare\.com' | head -1
```

■ 6.4 – Named Tunnel (URL קבוע על דומיין שלך)

ל-פרודקשן, אתה רוצה URL יציב כמו `https://api.yourdomain.com` במקום `hughes-random-words.trycloudflare.com` שמתחלף בכל `.restart`.

מה צריך לפני שמתחילים

- **דומיין** (מ-Namecheap, GoDaddy, Cloudflare Registrar, או כל רושם אחר). אפילו דומיין זול ב-\$10/שנה עובד.
- **הדומיין מנוהל ב-Cloudflare DNS**. אם רכשת ב-Cloudflare – אוטומטית. אם במקום אחר – צריך להוסיף את הדומיין ל-Cloudflare ולעדכן את ה-`nameservers`. הדרכה: developers.cloudflare.com/dns/zone-setups/full-setup.

מה לבחור: subdomain או root?

חומלץ: subdomain ייעודי, למשל `api.yourdomain.com` או `wa.yourdomain.com`.

) root (example.com		) subdomain (api.example.com	
	✗	✓	בידוד מהאתר הראשי
(דורש redirect)	✗	✓	יכול לחיות ביחד עם אתר wordpress / וכו' על השרת הראשי
רק לפרויקטים API-only		✓	נהוג בתעשייה

בדוגמאות מתחת אני אשתמש ב- `api.example.com` . תחליפו לדומיין שלכם.

שלב-אחר-שלב

1. התחברות ראשונית של cloudflared לחשבון שלך:

BASH העתק

```
cloudflared tunnel login
```

יוצא URL לדפדפן. פתחו אותו במחשב הראשי שלכם, התחברו ל-Cloudflare, ובחרו את הדומיין שלכם מהרשימה. אחרי האישור – `~/.cloudflared/cert.pem` נכתב על השרת.

2. יצירת tunnel בשם:

BASH העתק

```
cloudflared tunnel create wa-hub
```

הפלט יראה משהו כמו:

BASH העתק

```
Created tunnel wa-hub with id 4f7c9d3e-abcd-1234-5678-90abcdef1234
```

שמרו את ה-ID tunnel הזה. קוראים לו – במשך כל ה-instructions.

3. ניתוב DNS – מחבר את הסאב-דומיין ל-tunnel:

BASH העתק

```
cloudflared tunnel route dns wa-hub api.example.com
```

(החליפו `api.example.com` בסאב-דומיין שלכם). הפקודה הזאת יוצרת אוטומטית רשומת CNAME ב-Cloudflare שמצביעה לתעבורה לתוך ה-tunnel – אתם לא צריכים לפתוח את Cloudflare ולעשות זה ידנית.

אם תרצו עוד סאב-דומיינים על אותו tunnel (למשל גם `wa.example.com` או `api.example.co.il`), פשוט הריצו את הפקודה הזאת שוב לכל אחד.

4. קונפיגורציה של ה-tunnel – איזה traffic איפה ינתב:

BASH
העתק

```
mkdir -p /etc/cloudflared
```

BASH
העתק

```
nano /etc/cloudflared/config.yml
```

הדביקו (החליפו – ב-ID tunnel שלכם, ו- `api.example.com` בסאב-דומיין):

YAML
העתק

```
tunnel: <TID>
credentials-file: /root/.cloudflared/<TID>.json

ingress:
  - hostname: api.example.com
    service: http://127.0.0.1:3060
  - service: http_status:404
```

הסבר על הבלוק `ingress`:

- כל בקשה לכתובת `api.example.com` עוברת ל-Hub שלכם ב- `127.0.0.1:3060` ✓
 - כל בקשה אחרת מקבלת 404 (fallback ברירת מחדל – חובה!)
- שמרו (`Ctrl+0` , `Enter` , `Ctrl+X` ב-nano).

5. אם יש לכם כבר Quick Tunnel רץ כ-systemd – עצרו אותו:

BASH
העתק

```
systemctl disable --now cloudflared-wahub.service 2>/dev/null
```

(אחרת תהיו עם 2 tunnels שמתנגשים על אותו פורט.)

6. התקנה כ-systemd service של cloudflared הרשמית:

BASH
העתק

```
cloudflared service install
```

ה-CLI מתקין את עצמו כ-systemd עם השם `cloudflared.service`. הוא יקרא אוטומטית את `/etc/cloudflared/config.yml`.

7. הפעלה ובדיקה:

BASH
העתק

```
systemctl enable --now cloudflared
```

BASH
העתק

```
sleep 5 && systemctl status cloudflared --no-pager
```

צריך לראות `active (running)`. לוג שגיאות:

BASH
העתק

```
journalctl -u cloudflared -n 50 --no-pager
```

8. בדיקה ממחשב חיצוני:

BASH
העתק

```
curl https://api.example.com/healthz
```

צריך להחזיר `{ "ok": true, ... }`. אם כן – **הצלחתם!** ה-Hub שלכם זמין עכשיו ב- `https://api.example.com` באופן קבוע, מאחורי Cloudflare DDoS protection ו-SSL – הכל אוטומטי, חינם).

עדכוני הסביבה אחרי המעבר ל-Named Tunnel

- אם רשמתם **webhook ל-Hub** (דרך `PUT /api/instance/webhook`) – ההגדרה כבר נשמרה ב- `data/webhook.json` ושורדת `restart`; אין צורך לערוך `.env`. כדי לשנות אותה, הריצו `PUT` חדש.
- בכל מקום שהשתמשתם ב-URL הזמני של ה-Tunnel** (Base44 secrets, Bubble API config וכו') – תחליפו ל- `https://api.example.com`. זה כבר לא ישתנה.

טיפ: אותו tunnel יכול לשרת כמה אפליקציות במקביל – הוסיפו עוד זוגות `hostname` / `service` תחת `ingress` (כל subdomain לפורט שלו), והשאירו את `service: http_status:404` - כשורה אחרונה.

שלב 7 – שימוש מכל מערכת

ה-API שלך הוא **REST + JSON + Bearer auth** – הסטנדרט הכי משעמם בעולם. וזה היתרון. כל פלטפורמה שיודעת לעשות HTTP request מסוגלת לדבר איתו.

בכל הדוגמאות:

- `HUB_URL` = ה-URL הציבורי של ה-Tunnel שלך
- `HUB_TOKEN` = ה-Bearer מ- `.env`
- שמור אותם **רק בצד שרת / סודות**, לעולם לא ב-JS של דפדפן

■ Bubble – 7.1

ב-Bubble Plugins → **API Connector**:

WhatsApp Hub → תן שם **Add another API .1**

Private key in header **:Authentication .2**

Bearer **:Key value** · Authorization **:Key name .3**

:Add call .4

Send Text **:Name** ○

POST **:Method** ○

/api/messages/send/text **:URL** ○

JSON **:Body type** ○

{"to":"","text":""} **:Body** ○

parameters-כ text ו- to סמן את ○

. When Button "Send" is clicked → API call WhatsApp Hub Send Text **:workflow-ב**

■ **7.2** **Firebase Cloud Functions**

JAVASCRIPT
העתק

```
// functions/index.js
const { onRequest } = require("firebase-functions/v2/https");
const { defineSecret } = require("firebase-functions/params");

const HUB_TOKEN = defineSecret("HUB_TOKEN");
const HUB_URL = "https://your-tunnel.trycloudflare.com";

exports.sendWa = onRequest({ secrets: [HUB_TOKEN] }, async (req, res) => {
  const r = await fetch(`${HUB_URL}/api/messages/send/text`, {
    method: "POST",
    headers: {
      Authorization: `Bearer ${HUB_TOKEN.value()}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify(req.body),
  });
  res.status(r.status).json(await r.json());
});
```

. הזריקו את הסוד: `firebase functions:secrets:set HUB_TOKEN`

■ **7.3** **Make / n8n / Zapier**

שליחה:

Webhooks by Zapier / module HTTP Request (Make) / HTTP Node (n8n) **.1**

/api/messages/send/text **:URL .2**

POST **:Method .3**

Content-Type: application/json + Authorization: Bearer **:Headers .4**

`{"to":"+972...","text":"hi"}` :Body .5

קבלת webhook:

1. צרו Webhook ב-Make/n8n/Zapier → תקבלו URL

2. רשמו אותו ב-Hub:

BASH העתק

```
curl -X PUT -H "Authorization: Bearer $HUB_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"url\":"<your make/n8n webhook url>","events\":[\"message.incoming\"]}' \
  $HUB_URL/api/instance/webhook
```

כל הודעה נכנסת מופיעה בתוך Make/n8n כטריגר אוטומטי.

Node.js / Next.js – 7.4 ■

JAVASCRIPT העתק

```
// app/api/send-wa/route.js (Next.js App Router)
export async function POST(req) {
  const { to, text } = await req.json();
  const r = await fetch(`${process.env.HUB_URL}/api/messages/send/text`, {
    method: "POST",
    headers: {
      Authorization: `Bearer ${process.env.HUB_TOKEN}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify({ to, text }),
  });
  return Response.json(await r.json(), { status: r.status });
}
```


 PYTHON
העתק

```
import os, requests

HUB_URL = os.environ["HUB_URL"]
HUB_TOKEN = os.environ["HUB_TOKEN"]

def send_whatsapp(to: str, text: str):
    r = requests.post(
        f"{HUB_URL}/api/messages/send/text",
        headers={"Authorization": f"Bearer {HUB_TOKEN}"},
        json={"to": to, "text": text},
        timeout=15,
    )
    r.raise_for_status()
    return r.json()
```

:webhook (FastAPI) קבלת


 PYTHON
העתק

```
import hmac, hashlib, os
from fastapi import FastAPI, Request, HTTPException

app = FastAPI()
SECRET = os.environ["HUB_WEBHOOK_SECRET"]

@app.post("/wa/incoming")
async def incoming(req: Request):
    body = await req.body()
    got = req.headers.get("x-hub-signature", "")
    want = "sha256=" + hmac.new(SECRET.encode(), body, hashlib.sha256).hexdigest()
    if not hmac.compare_digest(got, want):
        raise HTTPException(401)
    event = await req.json()
    # ... your logic ...
    return {"ok": True}
```


PHP העתק

```
use Illuminate\Support\Facades\Http;

$r = Http::withToken(env('HUB_TOKEN'))
    ->post(env('HUB_URL') . '/api/messages/send/text', [
        'to' => $to,
        'text' => $text,
    ]);

return response()->json($r->json(), $r->status());
```

Google Sheets / Apps Script – 7.7 ■

JAVASCRIPT העתק

```
function sendWhatsApp(to, text) {
    const HUB_URL = PropertiesService.getScriptProperties().getProperty('HUB_URL');
    const HUB_TOKEN = PropertiesService.getScriptProperties().getProperty('HUB_TOKEN');
    return UrlFetchApp.fetch(`${HUB_URL}/api/messages/send/text`, {
        method: 'post',
        contentType: 'application/json',
        headers: { Authorization: `Bearer ${HUB_TOKEN}` },
        payload: JSON.stringify({ to, text }),
    }).getContentText();
}

// שלח הודעה לכל שורה ב-sheet
function bulkSend() {
    const rows = SpreadsheetApp.getActiveSheet().getDataRange().getValues();
    // ⚠️ אזהרה: שלחו אך ורק לנמענים שנתנו הסכמה מפורשת מראש (opt-in). שליחה לרשימה שנקנתה/נאספה
    // מפרה את חוק הספאם (סעיף 30א), GDPR/CAN-SPAM/TCPA, וכמעט בוודאות תגרום לחסימת המספר.
    rows.slice(1).forEach(([phone, message]) => sendWhatsApp(phone, message));
}
```

Shell / cron – 7.8 ■

BASH העתק

```
# /etc/cron.daily/wa-summary.sh
TOKEN=$(cat /etc/hub.token)
curl -X POST -H "Authorization: Bearer $TOKEN" \
    -H "Content-Type: application/json" \
    -d '{"to":"+972501234567","text":"Daily summary ready"}' \
    https://your-tunnel.trycloudflare.com/api/messages/send/text
```

התובנה: כמעט כל פלטפורמת no-code/low-code היום יודעת לעשות HTTP. ה-Hub שלך הופך לכל אחת מהן ל"ספק WhatsApp" באופן מיידי, בלי לקנות עוד תוסף או תשלום per-message.

שלב 8 – חיבור דרך מערכת vibe-coding

כל פלטפורמת "בנייה-לפי-פרומפט" (Base44, Bolt, Lovable, v0, Cursor, Claude Code, ...) יודעת לבנות לכם את החיבור ל-Hub מתיאור אחד. מדביקים פרומפט אחד, והיא בונה: סוד (token), פונקציית שליחה, endpoint ל-webhook שמאמת חתימת HMAC, וטבלת לוג הודעות.

8.1 – הפרומפט (פעם אחת, בכל מערכת)

טיפ אבטחה: אל תדביקו ערכי-סוד אמיתיים בצ'אט. רוב הפלטפורמות מזהות שמות-סוד ומבקשות אותם בדיאלוג מאובטח. אם לא – הגדירו אותם ידנית במנהל-הסודות של הפלטפורמה.

```
❏ רעיון WhatsApp connector for this app – no UI yet, just the plumbing.

SECRETS (store in the platform's secret manager, never hard-code):
WA_HUB_URL    = my Hub's public URL, e.g. https://api.example.com
WA_HUB_TOKEN  = bearer token for the Hub
WA_HUB_SECRET = HMAC secret for verifying incoming webhooks

BACKEND FUNCTION  sendWhatsApp(to, text):
  POST `${WA_HUB_URL}/api/messages/send/text`
  header: Authorization: Bearer ${WA_HUB_TOKEN}
  body:   { to, text }           // `to` = bare digits + country code, e.g. 972585802298

PUBLIC WEBHOOK ENDPOINT (the Hub calls it from the internet):
1. read the RAW request body as bytes BEFORE parsing JSON
2. verify header `x-hub-signature` equals
   "sha256=" + HMAC_SHA256(WA_HUB_SECRET, rawBody) // constant-time compare
   On edge/Deno runtimes use Web Crypto (`crypto.subtle`), NOT Node's Buffer.
3. if valid and event === "message.incoming":
   save { direction:"incoming", text, from, ts } to a `messages` table
4. ALWAYS return HTTP 200.

Don't build any pages yet – just the secrets, the two functions, and the log table.
I'll register the webhook URL on the Hub once it's deployed.
```

8.2 – מה לבנות עם זה (רעיונות, לא פרומפטים)

עכשיו שיש חיבור, בקשו מהמערכת לבנות מה שתרצו. כמה רעיונות:

- **CRM** עם כפתור "שלח WhatsApp" בכל כרטיס לקוח
- **דף צ'אט** בסגנון WhatsApp Web שקורא מטבלת ההודעות, מסונן לפי מספר

- **בוט שמגיב אוטומטית** – ראו שלב 11 (חיבור AI)
- **לוח בקרה:** כמה נשלח/התקבל השבוע + סטטוס חיבור בזמן אמת
- **פרסר הזמנות:** הודעה שמתחילה ב"הזמנה" → יוצרת רשומת Order ומתייגת את הצוות

8.3 – לרשום את ה-webhook ב-Hub

אחרי שה-endpoint עלה, קחו את ה-URL הציבורי שלו ורשמו אותו ב-Hub (זה מפעיל את הכיוון הנכנס):

BASH
העתק

```
TOKEN=$(grep HUB_TOKEN /srv/wa-hub-demo/.env | cut -d= -f2)
curl -X PUT -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"url":"https://your-app.example/webhook","events":
["message.incoming","message.outgoing"]}' \
  http://127.0.0.1:3060/api/instance/webhook
```

שני ה-events: `message.incoming` (הודעות שמגיעות אליכם) ו-`message.outgoing` (גם מה שאתם שולחים מהטלפון). ההגדרה נשמרת ב-`data/webhook.json` ושורדת restart (ראו טיפ #4).

אם המערכת שלכם היא Base44 (Deno runtime) – שלוש מלכודות נפוצות:

1. אמתו HMAC עם Web Crypto (`crypto.subtle`), לא עם `Buffer` / `timingSafeEqual` (קורס ב-500).
 2. כתיבה ל-DB מ-webhook אנונימי דרך `asServiceRole.entities.X` (לא `.entities.X`).
 3. `functions.invoke()` מחזיר axios wrapper – קראו `r.data.X` ולא `r.X`.
- דוגמה מלאה ומוכנה (TypeScript/Deno): [examples/base44/webhook-receiver.ts](#) ברפו.

שלב 9 – שליחה בטוחה: מניעת חסימה מ-WhatsApp

WhatsApp לא אוהבים שולחים אוטומטיים. הם משתמשים באלגוריתמים שמזהים התנהגות לא-אנושית ו**חוסמים מספרים**. שירותים מנוהלים מסחריים בדרך כלל מיישמים הגנות מסוג זה (השהיות, rate-limit-per-recipient, typing simulation וכו'). ה-Hub הזה לא כולל את ההגנות האלה מהקופסה. אבל זה לא בעיה – אפשר ליישם אותן בקוד שקורא ל-API שלנו. הנה המתכונים החשובים:

9.1 – השהייה רנדומלית בין הודעות

הסכנה: שליחת 100 הודעות בלי הפסקה תזוהה בוודאות כספאם. **הפתרון:** המתן 3-15 שניות רנדומליות בין הודעות.

PYTHON העתק

```
import random, time, requests

def send_safe(to, text):
    requests.post(f"{HUB_URL}/api/messages/send/text",
                  headers={"Authorization": f"Bearer {HUB_TOKEN}"},
                  json={"to": to, "text": text})
# רנדומלי 3-15 שניות – לחיקוי קצב כתיבה אנושי
time.sleep(random.uniform(3, 15))

# ⚠️ אזהרה: שלחו אך ורק לנמענים שנתנו הסכמה מפורשת מראש (opt-in). שליחה לרשימה שנקנתה/נאספה
# מפרה את חוק הספאם (סעיף 30א), GDPR/CAN-SPAM/TCPA, וכמעט בוודאות תגרום לחסימת המספר.
for recipient, message in customers:
    send_safe(recipient, message)
```

9.2 – סימולציית הקלדה (typing indicator)

לפני שליחת הודעה ארוכה, הראו "...typing" במשך כמה שניות. זה גורם להודעה להיראות אנושית. ה-Hub שלנו לא חושף endpoint לטייפינג ישירות (כי baileys עושה את זה אוטומטית בעבודה רגילה), אבל אפשר להוסיף השהייה בצד שלך:

PYTHON העתק

```
# המתנה כאילו אתה מקליד – 1~100ms
delay = min(len(text) * 0.1, 8) # מקסימום 8 שניות
time.sleep(delay)
send_safe(to, text)
```

9.3 – Rate limit לפי נמען

לעולם אל תשלחו ליותר מ-1 הודעה ל-30 שניות **לאותו מספר**, ואל תעברו את המכסה היומית שמתאימה לגיל המספר (ראו טבלת ה-warmup ב-9.4). שמרו טבלה של "מתי שלחתי לאחרון לכל מספר":

PYTHON העתק

```
last_sent = {} # number -> timestamp

def send_safe_per_user(to, text):
    now = time.time()
    if to in last_sent and now - last_sent[to] < 30:
        wait = 30 - (now - last_sent[to])
        time.sleep(wait)
    send_safe(to, text)
    last_sent[to] = time.time()
```

9.4 – Warmup למספרים חדשים

מספר שזה עתה נוצר/חובר לא מורגל לתעבורה. שלחו ממנו פחות הודעות בימים הראשונים:

ימים מההתחברות	מקסימום הודעות יוצאות ביום
1-3	10
4-7	30
8-14	100
15+	300+

9.5 – מתי להפסיק

WhatsApp נותנים אזהרות לפני חסימה – ההודעות שלך **לא מתקבלות עם וי כפול ירוק** אלא נשארות עם וי בודד שעות. אם זה קורה – **תפסיקו לשלוח מיד**, חכו 24 שעות, ושלחו במקצב נמוך משמעותית.

9.6 – סיכום: כללי אצבע

למה	כלל
זיהוי ספאם מיידי	אל תשלחו ל-100 מספרים תוך 10 דקות
אנושי	השאירו לפחות 30 שניות בין הודעות לאותו מספר
דיווחי ספאם מהמשתמשים – הסיבה #1 לחסימה	אל תשלחו לאנשים שלא ביקשו לקבל הודעות
מגבלה לא רשמית של WhatsApp	לא יותר מ-300 הודעות יוצאות ביום ממספר +1 ימים
סימן אזהרה לפני חסימה	אם וי בודד נשאר שעות – עצור!

רוצים מימוש בתוך ה-Hub? זה ב-roadmap (<docs/ROADMAP.md> ברפו). תרגישו חופשי לפתוח PR או issue ב-GitHub. בינתיים – הקוד שלך הוא המקום הנכון להוסיף את ההגנות (יותר גמיש לתפור לפי הצורך).

שלב 10 – אבטחה לפרודקשן

לפני שמשחררים את זה לעולם:

בדיקה	פעולה
HTTPS בלבד?	אם אתם משתמשים ב-Tunnel – כן, אוטומטית
Token חזק?	<code>openssl rand -hex 32</code> נותן 256 ביט – מספיק
?Rate limit	בקובץ <code>.env</code> : <code>RATE_LIMIT_PER_MIN=120</code> . עדכנו לפי הצורך
Webhook ?signed	הקוד מסמן HMAC-SHA256. הצד השני חייב לאמת
השרת מתעדכן?	<code>apt-get install -y unattended-upgrades</code>
ניטור?	<code>/healthz</code> על Uptime Robot + <code>journalctl -u wa-hub -f</code>
גיבוי?	תיקיית <code>/srv/wa-hub-demo/data/auth</code> מחזיקה את ה-session. ל-S3/R2 פעם ביום <code>rsync</code>
סודות לא ב-git?	<code>.gitignore</code> כבר חוסם <code>.env</code> . וודאו שלא הוספתם בטעות
תקרת זיכרון?	הוא יחזיר את השירות אוטומטית OOM-כולל <code>MemoryMax=512M</code> ב <code>systemd unit</code>

10.1 – סיבוב Token

אם ה-Token דלף או חשדתם:

BASH
העתק

```
NEW=$(openssl rand -hex 32)
sed -i "s/^HUB_TOKEN=.* /HUB_TOKEN=$NEW/" /srv/wa-hub-demo/.env
systemctl restart wa-hub
echo "$NEW"
```

עדכנו ב-Base44 secrets (או היכן שאתם משתמשים) – וזהו. 30 שניות downtime.

10.2 – זיכרון לטווח ארוך

Baileys שומר היסטוריית chats ב-RAM. ב-instance עם מעט תעבורה (פחות מ-100 הודעות/יום) זה לא בעיה. ב-instance פעיל יותר, הזיכרון עלול לטפס.

הגנות שכבר במקום:

- `syncFullHistory: false` ב- `src/baileys/socket.js` – אנחנו לא מורידים את כל ההיסטוריה הישנה
- `markOnlineOnConnect: false` – חוסכים traffic של "אונליין" notifications
- `MemoryMax=512M` ב- `systemd unit` – אם בכל זאת זה תופח מעבר, `systemd` יחזיר את השירות (Baileys ייחבר מחדש)

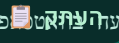
אם יש לכם volume גבוה במיוחד (אלפי הודעות ביום): שקלו להפחית את ה-cache ב-Baileys, או לפצל ל-multiple instances.

שלב 11 – חיבור מודל AI

אם בעבר בניתם בוט Node עם Baileys והטמעתם בתוכו ישירות את ה-API של Gemini/GPT – שימו לב לשינוי התפיסתי. כآن הארכיטקטורה **מפצלת לשתי שכבות**:

- ה-Hub = צינור HTTP ↔ WhatsApp בלבד. הוא **בכוונה לא יודע מה זה AI**.
 - ה"מוח" שלכם = הקוד שמקבל הודעה, חושב (קורא ל-AI), ומחזיר תשובה.
- ה-AI **לא נכנס לתוך ה-Hub** – הוא יושב בקוד שמקבל את ה-webhook. זה בדיוק המקום שבו פעם הטמעתם את ה-API של גוגל, רק שעכשיו הוא מנותק משכבת התקשורת.

11.1 – הזרימה: איפה ה-AI נכנס

הערה

1. מישור שולח הודעה

2. ה-Hub קולט ויורה `event: "message.incoming"` ל-webhook

3. ה-webhook מגיע לפונקציה שלכם ← כאן יושב ה-AI

4. הפונקציה קוראת ל-Gemini/Claude/GPT עם הטקסט

5. הפונקציה שולחת את התשובה חזרה: `POST {HUB_URL}/api/messages/send/text`

6. ה-Hub שולח את התשובה לוואטסאפ

11.2 – שלושה מקומות לשים את הפונקציה

מקום	מתי	הערה
פונקציית שרת בפלטפורמה (Base44 / Firebase)	"פונקציה באתר", בלי שרת נוסף	ה-AI רץ בתוך ה-backend function שמקבלת את ה-webhook
שרת Node משלכם	הכי קרוב למה שעשיתם עד היום	במקום <code>import Baileys</code> – <code>fetch</code> ל-Hub. ה-Hub מחליף את Baileys
כלי אוטומציה (Make / n8n)	בלי קוד	חזרה node HTTP → node AI → node webhook ל-Hub

11.3 – דוגמה: שרת Node עם Gemini

צ'אטבוט AI מלא. שימו לב – הקוד הזה **לא מכיר את Baileys בכלל**, רק מדבר HTTP מול ה-Hub:

JAVASCRIPT העתק

```
import express from "express";
import { createHmac } from "node:crypto";

const app = express();
const SECRET = process.env.WA_HUB_SECRET; // = WEBHOOK_SECRET מה-.env
const HUB_URL = process.env.WA_HUB_URL; // https://api.example.com
const HUB_TOKEN = process.env.WA_HUB_TOKEN; // = HUB_TOKEN מה-.env
const GEMINI_KEY = process.env.GEMINI_API_KEY;

app.use(express.raw({ type: "*/*" })); // גוף גולמי - צריך אותו לאימות החתימה

app.post("/wa", async (req, res) => {
  const body = req.body.toString("utf8");
  const expected = "sha256=" + createHmac("sha256", SECRET).update(body).digest("hex");
  if (req.get("x-hub-signature") !== expected) return res.status(401).send("bad sig");

  res.sendStatus(200); // וממשיכים ברקע, (אחרת ינסה שוב) Hub-עונים מיד ל

  const { event, data } = JSON.parse(body);
  if (event !== "message.incoming" || data.type !== "text") return;

  const reply = await askGemini(data.text); // ← כאן ה-AI

  await fetch(`${HUB_URL}/api/messages/send/text`, {
    method: "POST",
    headers: { "Authorization": `Bearer ${HUB_TOKEN}`, "Content-Type": "application/json" },
    body: JSON.stringify({ to: data.from, text: reply }),
  });
});

async function askGemini(userText) {
  const r = await fetch(
    `https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent?key=${GEMINI_KEY}`,
    { method: "POST", headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ contents: [{ parts: [{ text: userText }] }] })
  );
  const j = await r.json();
  return j?.candidates?.[0]?.content?.parts?.[0]?.text ?? "סליחה, לא הצלחתי לענות כרגע";
}

app.listen(8080, () => console.log("AI brain on :8080"));
```

רושמים את ה- `/wa` כ-webhook ב-Hub (`PUT /api/instance/webhook`), בדיוק כמו בשלב 7.3. רוצים Claude במקום Gemini? החליפו רק את `askGemini` בקריאה ל- `api.anthropic.com/v1/messages` – כל השאר זהה.

11.4 – דוגמה: פונקציית Base44 (בלי שרת נוסף)

יש לכם כבר שלד מוכן ב- `examples/base44/webhook-receiver.ts` שמחזיר "echo". החליפו את ה-echo בקריאה ל-AI:

TYPESCRIPT
העתק

```
if (event.event === "message.incoming" && event.data.type === "text") {
  const reply = await askGemini(event.data.text as string); // מ-askGemini 11.3 אותה
  await fetch(`${Deno.env.get("WA_HUB_URL")}/api/messages/send/text`, {
    method: "POST",
    headers: { "Authorization": `Bearer ${Deno.env.get("WA_HUB_TOKEN")}`, "Content-Type":
"application/json" },
    body: JSON.stringify({ to: event.data.from, text: reply }),
  });
}
```

אפשר לדחוף את ה-AI ישיר לתוך ה-Hub? טכנית כן (זה Node פתוח). אבל **לא מומלץ** – זה מחזיר אתכם לערבוב שניסינו לפרק. השאירו את ה-Hub "טיפש" וצינור-בלבד, ואת החוכמה אצלכם. כך עדכון של ה-Hub לא נוגע בלוגיקת ה-AI, ואפשר להחליף מודל בלי לגעת בשכבת הוואטסאפ.

טיפים מהשטח – דברים שלא תמצאו בתיעוד

1 – Baileys 7 ומספרי LID

החל מ-2025, WhatsApp מציגים **LID (Logical ID)** – מזהה ייחודי לכל משתמש שלא חושף את מספר הטלפון שלו. זה נועד למנוע harvesting של מספרים על-ידי mass-scraping.

המחלקה
לפני: `phone` ← `fromLid = false` `fromNumber = "972501234567"`
אחרי: `LID` ← `fromLid = true` `fromNumber = "8362502693023"`, מספר 13 ספרות מוזר

ה-Hub מטפל בזה אוטומטית: קוד `extractPhone()` מנסה לחלץ מספר טלפון אמיתי דרך `senderPn` → `participantPn` → `remoteJidAlt`. אם אין – חוזר ל-LID. שדה `fromLid: true/false` מאותת לכם מהו הזיהוי המקורי.

אם אתם מסננים לפי `fromNumber` ורואים שמסננים יותר מדי – בדקו `fromLid` ב-payload.

2 – Base44 functions.invoke מחזיר axios wrapper

JAVASCRIPT
העתק

```
const r = await base44.functions.invoke("foo", {});
// r = { data: <actual response>, status: 200, headers: {...} }
```

תקראו `r.data.X` ולא `r.X`. אם הקוד שלכם מתנהג כאילו פונקציה נכשלת אבל ה-network tab מראה OK 200 – זה הסיבה.

■ #3 Base44 entities ברירת מחדל owner-only

ללא בלוק `rls` בסכמה, רק היוצר רואה את הרשומה. webhooks ש-anon נכנסות לא יוכלו לקרוא, גם דרך `asServiceRole` במצב מסוים. הגדירו `rls: { read: true, create: true, ... }` או עברו את כל הקריאות דרך `functions` עם `service role`.

■ #4 Webhook config נשמר אוטומטית (לא נמחק ב-restart)

נשמר לדיסק ב- `data/webhook.json` ושורד restart – והקובץ הזה גובר על מה שב- `.env`. כלומר אחרי PUT אחד, אין צורך לערוך `.env`.
ה- `WEBHOOK_EVENTS` / `WEBHOOK_URL` ב- `.env` משמשים רק כברירת מחדל אם מעולם לא עשיתם PUT (כלומר אין עדיין `data/webhook.json`):

BASH
העתק

```
WEBHOOK_URL=https://your-receiver.com/wa  
WEBHOOK_EVENTS=message.incoming
```

לשנות webhook אחרי PUT: הריצו PUT חדש, או מחקו את `data/webhook.json` כדי לחזור לברירת המחדל מ- `.env`.

■ #5 Quick Tunnel מתחדש בכל restart

מצוין לדמו, אסון לפרודקשן. עברו ל-Named Tunnel עם דומיין משלכם ברגע שאתם עוברים מ-experiment ל-production.

■ #6 Linked Device timeout – 14 יום

אם הטלפון הראשי לא היה online במשך 14 יום, WhatsApp מנתקים את כל ה-Linked Devices. תצטרכו פיירינג QR מחדש. אם אתם רוצים שזה יחזיק יותר – ודאו שהטלפון הראשי מחובר לרשת לפחות פעם ב-13 יום.

■ #7 base44 0.0.52-bug + Node.js 24 + Windows = נתיב עברי

על Windows עם שם משתמש עברי (כמו "נעם ניסן"), הגרסה האחרונה של חבילת `base44` ב-npm נכשלת ב- `cp` של `assets`. הפתרון:

BASH
העתק

```
npm install --save-dev base44@0.0.51
```

נהוג להתקין global גם:

BASH
העתק

```
npm install -g base44@0.0.51
```

(זה יעלים כשהם יתקנו את ה-bug, אבל נכון לכתיבת המדריך הזה הבעיה קיימת.)

■ #8 – apt-get zombie על שרתי cloud

ראיתי שרת Hetzner שבו `apt-get update` נשאר רץ במשך **11 ימים**, חוסם את כל ניסיונות העדכון של `unattended-upgrades`. אם עדכוני אבטחה לא תופסים – בדקו:

BASH העתק

```
ps -ef | grep apt-get | grep -v grep
# אם רואים תהליך מ-3+ ימים – הרגו:
kill -9 <PID>
rm -f /var/lib/dpkg/lock-frontend /var/lib/dpkg/lock /var/cache/apt/archives/lock
apt-get update
```

פתרון בעיות נפוצות

▼ השירות מופעל אבל QR לא מופיע

BASH העתק

```
journalctl -u wa-hub -n 50 --no-pager
```

חפשו `QR generated`. אם לא מופיע – בדקו שיש חיבור לאינטרנט:

BASH העתק

```
curl -sS https://web.whatsapp.com | head -1
```

▼ "already paired" אבל בטלפון לא רואים את ההתקן

ה-Hub חושב שהוא מחובר אבל בעצם לא. אפסו:

BASH העתק

```
curl -X POST -H "Authorization: Bearer $TOKEN" \
http://127.0.0.1:3060/api/instance/logout
```

ובקשו QR חדש.

▼ הודעות לא נשלחות – not_connected

ב-95% מהמקרים זה כי הטלפון לא היה מחובר לאינטרנט יותר מ-14 יום, וה-session פג. הריצו את תהליך הפיירינג מחדש מהשלב 5.

▼ Cloudflare Tunnel נופל

BASH
העתק

```
systemctl status cloudflared-wahub # Quick tunnel
systemctl status cloudflared      # Named tunnel
journalctl -u cloudflared-wahub -n 50 --no-pager
```

לרוב זה שינוי ב-URL (ב-Quick Tunnel) – restart ייצור URL חדש. שמרו את ה-URL החדש ועדכנו בכל מקום שהשתמשתם בו.

▼ השירות נופל ב-core-dump • status=31/SYS

זה systemd seccomp filter – SIGSYS חוסם syscall ש-Node 20+ צריך (`io_uring` , `clone3`). לוג יראה משהו כמו:

BASH
העתק

```
wa-hub.service: Main process exited, code=dumped, status=31/SYS
wa-hub.service: Failed with result 'core-dump'.
```

תיקון: הסירו את ה-SystemCallFilter דרך drop-in:

BASH
העתק

```
mkdir -p /etc/systemd/system/wa-hub.service.d
printf '[Service]\nSystemCallFilter=\n' > /etc/systemd/system/wa-hub.service.d/syscall-
fix.conf
systemctl daemon-reload && systemctl restart wa-hub.service
```

שאר ההגנות (`NoNewPrivileges` , `ProtectSystem=strict`) וכו' נשארות בתוקף.

Webhook לא נכנס ל-Base44 (OK 200 בלוג של Hub, אבל ה-entity ריק)

1. ראו את הלוגים של הפונקציה: `npx base44 logs --function whatsappWebhook --limit 20`
2. אם רואים שגיאת RLS – וודאו שב- `whatsappmessage.jsonc` יש `rls: { create: true, read: true, ... }`
3. אם רואים שגיאת חתימה – בדקו ש- `WA_HUB_SECRET` ב-Base44 secrets זהה לחלוטין ל- `WEBHOOK_SECRET` ב- `.env` של השרת

הודעות נכנסות יש, אבל fromNumber הוא LID

ראו "טיפ #1" למעלה. השדה `fromLid: true` מאותת שזה LID. אם רוצים את מספר הטלפון האמיתי – אין דרך, WhatsApp לא חושפים את זה. תזהו לקוחות לפי `chat` (ה-JID של השיחה) שיציב לאורך זמן.

הצעדים הבאים

- **רוצים multi-tenant?** הוסיפו טבלת tenants ל-DB → לכל לקוח `instance_id` ו-port נפרד. אפשרות נוספת: להריץ `wa-hub-demo` כמה פעמים בקונטיינרים על אותו שרת.
- **רוצים AI אוטומטי?** ראו שלב 11 – צ'אטבוט מלא תוך 30~ שורות.
- **רוצים התראות לצוות?** ב- `/api/instance/webhook` אפשר להגדיר Slack/Discord webhook במקום (או בנוסף ל-) `Base44`.
- **רוצים dashboard?** ה-WebSocket ב- `:3061` משדר כל אירוע בזמן אמת – חברו לאפליקציית React/Vue/Svelte. ⚠️ ה-WS דורש את ה-token ב-query (`?token=...`), וטוקנים ב-URL דולפים ללוגים של Cloudflare ולהיסטוריית הדפדפן – השתמשו ב-token ייעודי/קצר-מועד ל-dashboards (לא ב- `HUB_TOKEN` הראשי), ושרתו את ה-WS רק מאחורי ה-Tunnel המוצפן.
- **רוצים סיכומים יומיים?** `cron` + `curl` ב- `/etc/cron.daily/`.

כלים שכדאי להכיר

- **Baileys** – הספרייה שעושה את הקסם של WhatsApp Web
- **Cloudflared** – Tunnel
- **Express** – REST framework
- **Hetzner Cloud** – VPS מומלץ
- **Uptime Robot** – חינם לניטור 50 endpoints
- **Cloudflare R2** – אחסון זול לגיבוי `data/auth`

שאלות נפוצות

- Q: זה חוקי? A:** צריך להפריד בין שתי שאלות. *חוקית לפי חוק* – שימוש לגיטימי בחשבון שלכם לתקשורת מבוקשת אינו עבירה. *מותר לפי תנאי השימוש של WhatsApp/Meta?* – לא: זו שיטה לא-רשמית (מתחזים ל-Linked Device), ו-Meta שומרת לעצמה את הזכות לחסום מספרים, במיוחד כאלה שמתנהגים כספאם. לשליחה בנפח גבוה / מסחרית – השתמשו ב-WhatsApp Cloud API הרשמי של Meta. כאן: שלחו רק הודעות שמישהו ביקש (ראו פרק "שליחה בטוחה").
- Q: השרת יעמוד בעומס? A:** CX23 / CAX11 (4GB RAM) טובים עד ~50 הודעות לשנייה. אם אתם מצפים ליותר – שדרגו ל-CX33 / CAX21 (8GB), או הריצו כמה instances.
- Q: מה קורה אם Hetzner נופל? A:** ב-12 חודשים אחרונים – uptime ~99.95%. לפרודקשן רצינית שקלו multi-region או fallback cloud-ל אחר.
- Q: מה הסיכון שיחסמו לי את המספר? A:** נמוך אם אתם משתמשים בו כמו אדם רגיל – תגובות, סיכומים, ורק למי שביקש. גבוה אם אתם שולחים cold outreach או נפח גבוה. היצמדו למגבלות בפרק "שליחה בטוחה" (כתקרה גסה: עד ~300 הודעות יוצאות ביום למספר מבוסס, והרבה פחות למספר חדש – ראו טבלת ה-warmup).
- Q: יש Web Crypto במקום node:crypto? A:** כן – ראו דוגמה ב-8.2 (ה-prompt של whatsappWebhook) לסביבות שלא תומכות ב-Node modules (Cloudflare Workers, Vercel Edge, Base44 Deno וכו'). שני הפתרונות עובדים זהה לחתימת HMAC.
- Q: אפשר להשתמש בקוד מסחרית? A:** כן. רישיון MIT – תפרק, תשנה, תפרסם, תמכור. אם תרגישו בנוח, תזכירו את הפרויקט המקורי.

נספח – מדריך חירום מהיר

חמש התקלות הכי נפוצות – והפקודה האחת שמתקנת כל אחת. שמרו את העמוד הזה בהישג יד.

תקלה	סימן בשטח	תיקון מיידי (שורה אחת על השרת)
ה-Hub לא רץ	<code>curl 127.0.0.1:3060/healthz</code> מחזיר connection refused	<code>alctl -u wa-hub -n 20 --no-pager</code>
URL של Tunnel השתנה	בקשות חיצוניות מקבלות 530/1033 פתאום	<code>+\.trycloudflare\.com' head -1</code>
QR פג תוקף לפני שסרקתם	בטלפון "קוד לא תקין"	חזרו על 2 הפקודות ב-5.2S (Enter ב-Enter)
Device was logged out	הלוג צועק <code>loggedOut</code> , אין QR חדש	<code>7.0.0.1:3060/api/instance/logout</code>
השירות נופל ב-	systemd: <code>code=dumped, status=31/SYS</code>	<code>Load && systemctl restart wa-hub</code>

■ צ'קליסט 60-שניות לבדיקת תקינות

1. SSH פתוח לשרת בטרמינל אחד, PowerShell מקומי בטרמינל שני.
 2. `TOKEN=$(grep HUB_TOKEN /srv/wa-hub-demo/.env | cut -d= -f2)` כבר רץ ב-SSH.
 3. `curl -sS http://127.0.0.1:3060/healthz` מחזיר `{"ok":true}`.
 4. ה-Tunnel URL מועתק לקליפבורד (`journalctl -u cloudflared-wahub | grep trycloudflare | head -1`).
 5. הטלפון פתוח ב-WhatsApp → הגדרות → מכשירים מקושרים, מוכן לסרוק.
- אם 1-5 ✓ – אפשר להתחיל. אם משהו אדום – שורה רלוונטית בטבלה למעלה תסדר את זה בפחות מ-30 שניות.

יש לכם עכשיו תשתית WhatsApp עצמאית מקצועית.

קוד פתוח: github.com/Noam13-w/wa-hub-demo